

Brno University of Technology
Faculty of Information Technology

Image Processing (in English) (ZPOe 25/26)

Application of Basic Image Processing Filters on Human Face Images

Project Report

Adar Otieno	`xotiena00`
Deniz Aksoy	`xaksoyd00`
Eyobed Awel Nuri	`xnuriey00`

May 9, 2026

Contents

1 Introduction **3**

2 Compilation and Running **3**

3 Implementations **3**

3.1 Eyobed's Filters . . . 3

3.1.1 Original Input Images . . . 3

3.1.2 Histogram Equalization . . . 4

3.1.3 Lens Distortion . . . 4

3.1.4 Posterize . . . 5

3.1.5 Vignette Blur . . . 6

3.1.6 Discussion . . . 7

3.2 Adar's Filters . . . 7

3.3 Deniz's Filters . . . 7

3.3.1 Original Input Images . . . 7

3.3.2 Threshold Filter . . . 8

3.3.3 Diffuse Filter . . . 8

3.3.4 Crosshatch Filter . . . 9

3.3.5 Hue Rotate Filter . . . 10

4 Conclusion **10**

1 Introduction

The students have decided to rework the project by removing previously proposed trivial filters such as sharpen and replaced them with more sophisticated ones as per the suggestions of the professor. We developed 12 filters from different categories in total, 4 filters per student.

2 Compilation and Running

The project uses CMake for build management. Standard compilation and execution steps:

```
mkdir build && cd build
cmake .. && make
./project
```

The application expects input images in the `data/` directory. The GUI features trackbars for image/filter selection and a side-by-side visualization window (800x600) with a metadata panel for filter descriptions. If you can not see the windows, drag them around to expose them.

3 Implementations

This section presents the filters developed by each student. Each student individually developed 4 filters, only the GUI is shared by the students.

3.1 Eyobed's Filters

3.1.1 Original Input Images

The following four images (Figure 1) serve as the baseline for all subsequent filter demonstrations in this section.



(a) Source 1 (b) Source 2 (c) Source 3 (d) Source 4

Figure 1: Original unprocessed input images used for Eyobed's filter evaluations.

3.1.2 Histogram Equalization

Histogram Equalization enhances image contrast by redistributing the brightness values. To prevent unwanted color shifting, the algorithm operates in the YCrCb color space, applying the equalization only to the luminance channel before converting back to BGR.

The algorithm calculates the histogram frequencies, the Probability Density Function (PDF), and the Cumulative Distribution Function (CDF). The PDF for a pixel intensity i is calculated as:

$$PDF(i) = \frac{\text{count}(i)}{\text{total pixels}}$$

The CDF is then calculated as the cumulative sum of the PDF:

$$CDF(i) = \sum_{j=0}^{i} PDF(j)$$

Finally, the new remapped pixel value is computed by scaling the CDF to the 0-255 range:

$$\text{Output}(x, y) = \text{round}(CDF(I(x, y)) \times 255)$$



(a) Result 1 (b) Result 2 (c) Result 3 (d) Result 4

Figure 2: Results after applying the Histogram Equalization filter.

3.1.3 Lens Distortion

The filter uses an inverse geometric mapping to simulate lens distortion. For each output pixel (x, y), the source coordinate (s_x, s_y) is calculated through three steps listed below:

1. Normalization: Coordinates are centered and scaled by $R = \sqrt{c_x^2 + c_y^2}$:

$$d_x = \frac{x - c_x}{R}, \quad d_y = \frac{y - c_y}{R}$$

2. Distortion: A radial factor is computed using coefficient k:

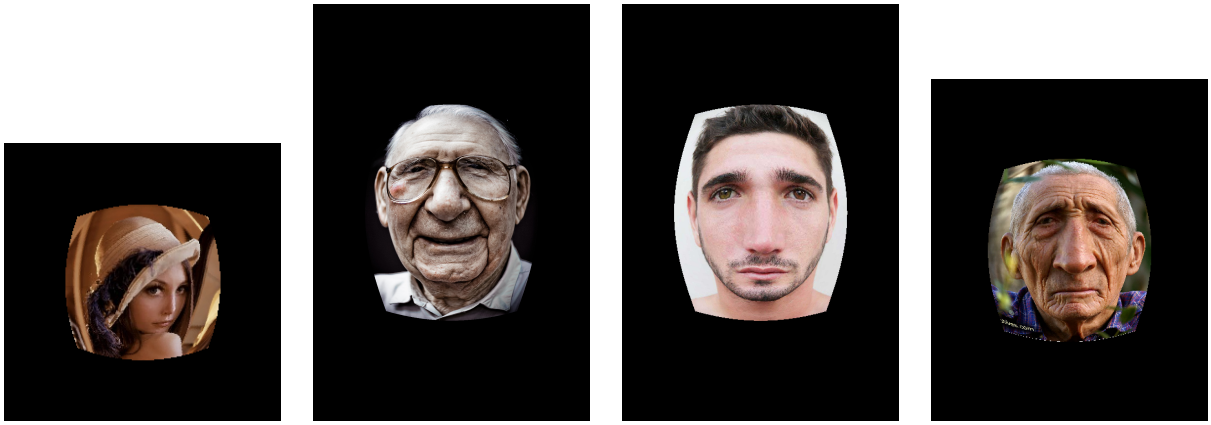
$$\text{dist} = 1 + k(d_x^2 + d_y^2)$$

Positive k produces barrel distortion, while negative k produces pincushion effects.

3. Mapping: Coordinates are scaled back to source pixel space:

$$s_x = d_x \cdot \text{dist} \cdot R + c_x, \quad s_y = d_y \cdot \text{dist} \cdot R + c_y$$

Pixels are sampled using nearest-neighbor interpolation. The computational complexity is $O(W \times H)$.



(a) Result 1 (b) Result 2 (c) Result 3 (d) Result 4

Figure 3: Results after applying the Lens Distortion filter ($k = 5.0$).

3.1.4 Posterize

Reduces color depth by quantizing each channel into $L = 8$ discrete levels. A 256-entry LUT is computed once to improve performance.

1. Step Calculation: The spacing between levels:

$$\text{step} = \frac{255}{L - 1} \approx 36.4$$

2. LUT Construction: Each intensity i is snapped to the nearest level index, then scaled back:

$$\text{level}_i = \text{round}\left(\frac{i}{\text{step}}\right), \quad \text{LUT}[i] = \text{round}(\text{level}_i \times \text{step})$$

3. Pixel Remapping: Each BGR channel is independently mapped:

$$\text{Output}_c(x, y) = \text{LUT}[\text{Input}_c(x, y)], \quad c \in \{B, G, R\}$$

Complexity: $O(W \times H)$ for the pixel pass, plus $O(256)$ for LUT construction.



(a) Result 1 (b) Result 2 (c) Result 3 (d) Result 4

Figure 4: Results after applying the Posterize filter ($L = 8$ levels).

3.1.5 Vignette Blur

Softens edges while keeping the center sharp. Implemented as a two step process: a 15×15 box blur followed by a radial blend.

1. **Box Blur:** For interior pixels (7-pixel border excluded), compute the 15×15 mean kernel convolution.

$$\text{Blur}(x, y) = \frac{1}{225} \sum_{dy=-7}^{7} \sum_{dx=-7}^{7} \text{Input}(x + dx,\ y + dy)$$

2. **Radial Mask:** Normalize the distance from center (c_x, c_y) by the corner radius R:

$$d = \frac{\sqrt{(x - c_x)^2 + (y - c_y)^2}}{R}, \quad d \in [0, 1]$$

The center maps to 0 (no blur) and corners to 1 (full blur).

3. **Blending:** Linearly interpolate between original and blurred using d as the weight:

$$\text{Output}(x, y) = \text{Input}(x, y) \cdot (1 - d) + \text{Blur}(x, y) \cdot d$$

Complexity: $O(W \times H)$ since the blur and blend are computed in a single pass.



(a) Result 1 (b) Result 2 (c) Result 3 (d) Result 4

Figure 5: Results after applying the Vignette Blur filter using a 15x15 manual kernel.

3.1.6 Discussion

The author of this section studied about implementation of histogram eqalization,lens distortion, posterization, and vignette blur filters. He believes that these filters are good demonstrations of concepts discussed in class. Additional techniques like look up tables helped to take into consideration the performance aspect of image processing.

3.2 Adar's Filters

Adar's contributions focus on manual pixel manipulation using native loops, avoiding optimized OpenCV functions for educational purposes.

- **Relief Emboss:** Creates a 3D gray relief texture by subtracting the bottom-right neighboring pixel from the top-left neighbor and adding an offset of 128 to center the result.
- **Dodge:** Implements the classic Color Dodge blend formula: `(pixel * 255) / max(1, 255 - pixel)`. This blows out the highlights and drastically increases contrast.
- **Swirl:** Warps the image into a vortex by converting Cartesian coordinates to Polar coordinates, applying a nonlinear rotational offset based on the radius, and converting back to sample the pixel.
- **Bloom:** Extracts bright pixels (intensity > 180) and applies a brute-force nested 7x7 loop to blur them. The resulting glow is then added directly back to the original image.

3.3 Deniz's Filters

Deniz's filters focus on fundamental image processing operations implemented through direct pixel-by-pixel matrix iteration. The implemented filters demonstrate concepts such as binary segmentation, stochastic distortion, procedural rendering, and color-space manipulation.

3.3.1 Original Input Images

The following images were selected as the common input dataset for evaluating the implemented filters.



(a) Input 1 (b) Input 2 (c) Input 3 (d) Input 4

Figure 6: Original input images used for evaluating Denis's image processing filters.

3.3.2 Threshold Filter

The Threshold filter converts the image into a binary black-and-white representation using grayscale luminosity computation and threshold comparison. It is commonly used in OCR systems, segmentation, and preprocessing pipelines.

$$L = 0.299R + 0.587G + 0.114B$$

$$\text{Output} = \begin{cases} 255, & \text{if } L > T \\ 0, & \text{otherwise} \end{cases}$$

The algorithm processes each pixel independently with linear complexity:

$$O(W \times H)$$



(a) Result 1 (b) Result 2 (c) Result 3 (d) Result 4

Figure 7: Results after applying the Threshold filter.

3.3.3 Diffuse Filter

The Diffuse filter creates a frosted-glass effect by randomly replacing each pixel with a neighboring pixel within a selected radius.

$$(x', y') = (x + r_x,\ y + r_y)$$

The filter produces stochastic local distortions while maintaining linear computational complexity:

$$O(W \times H)$$



(a) Result 1 (b) Result 2 (c) Result 3 (d) Result 4

Figure 8: Results after applying the Diffuse filter.

3.3.4 Crosshatch Filter

The Crosshatch filter simulates sketch-style rendering using procedural diagonal line generation based on pixel coordinates.

$$(x + y) \bmod n = 0$$

Darker image regions generate denser hatch patterns, creating a pen-and-ink drawing effect.

$$O(W \times H)$$



(a) Result 1 (b) Result 2 (c) Result 3 (d) Result 4

Figure 9: Results after applying the Crosshatch filter.

3.3.5 Hue Rotate Filter

The Hue Rotate filter modifies image colors by rotating the Hue channel in HSV color space.

$$H' = (H + \Delta H) \bmod 180$$

The implemented version applies a 60° hue shift while preserving brightness and structural information.

$$O(W \times H)$$



(a) Result 1 (b) Result 2 (c) Result 3 (d) Result 4

Figure 10: Results after applying the Hue Rotate filter.

4 Conclusion

This project successfully demonstrates the application of both fundamental and advanced image processing techniques. The students also estimated the computational complexity of each filter thanks to the knowledge they acquired in the course TINe. Moreover, we also learned the importance of using appropriate data structures and algorithms for image processing. The authors would like to acknowledge the utility of AI tools such as ChatGPT in the realization of this project in a reasonable and responsible manner.

References

[1] Ma, L., Chen, Y., & Moore, K. L. (2003). *A New Analytical Radial Distortion Model for Camera Calibration*. arXiv preprint cs/0307046.